

UNIVERSITÀ DEGLI STUDI DI PADOVA

Dipartimento di Fisica e Astronomia “Galileo Galilei”

Corso di Laurea in Fisica

Tesi di Laurea

Titolo

Relatore

Prof./Dr. Nome Cognome

Correlatore

Prof./Dr. Nome Cognome

Laureando

Alberto Stocco

Anno Accademico xxxx/xxxx

Indice

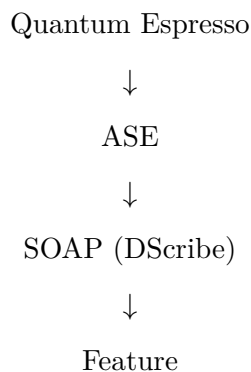
1	Struttura di lavoro	1
1.0.1	Dataset A	1
1.1	SOAP	1
2	Discussione teorica sulle librerie python utilizzate	5
2.1	ASE	5
2.2	DScribe	5

Capitolo 1

Struttura di lavoro

Lo scopo che ci poniamo per questo lavoro è quello di partire da simulazioni di dinamica molecolare (MD) eseguite con Quantum Espresso (QE), simulazioni che sono costose sia dal punto di vista computazionale che temporale, ed arrivare ad un modello di machine learning (ML) che sia in grado di svolgere una simulazione il più possibile accurate e con tempi di esecuzione sensibilmente minori.

Il processo che andiamo a seguire è descritto in modo schematico qui sotto:



Schema 1.1: Struttura schematica del processo di creazione dei feature vector

1.0.1 Dataset A

Questo dataset consiste di una simulazione di MD eseguita con QE per una struttura cristallina costituita da Ta_2O_5 , pentossido di ditantalio (a cui ci riferiamo anche come tantala). Questi dati seguono il processo descritto sopra per generare un vettore di *feature*, per una descrizione di questo si veda la sezione successiva. Questo dataset è composto da una serie di file di QE che costituiscono la dinamica molecolare di una struttura di tantala a circa 3000K.

Usando la libreria ASE abbiamo generato una rappresentazione della cella atomica dai dati di QE analizzati, essendo una simulazione di dinamica molecolare, riportiamo solo uno degli step, a cui diamo il nome di frame. Riportiamo di seguito questa immagine, disegnata tramite matplotlib 1.1.

1.1 SOAP

In questa sezione ci poniamo come obiettivo quello di dare una breve ma decente spiegazione, per quanto possibile completa e chiusa, del metodo SOAP (Smooth Overlap of Atomic Positions). Come indicato sopra questa procedura è implementata tramite DScribe.

Il punto fondamentale per comprendere la necessità di utilizzare una procedura come SOAP risiede nel concetto di invarianze, è infatti comune per le leggi della fisica essere invarianti per determinate proprietà, come lo possono essere le rotazioni, le traslazioni spaziali e lo scambio tra atomi uguali.

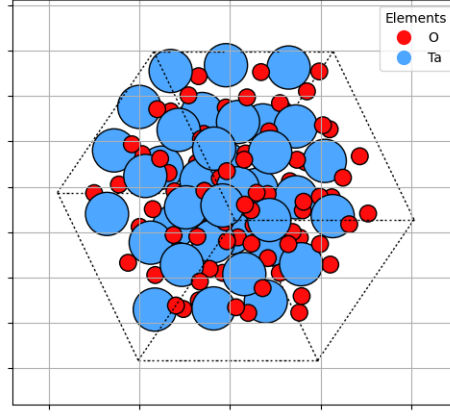


Figura 1.1: Rappresentazione generata tramite ASE della struttura atomica del dataset A

Queste simmetrie emergono in modo naturale dalle leggi utilizzate, ma come un fisico deve impararle e con fatica ricavarle anche un modello di ML necessiterebbe di imparare queste simmetrie ogni singola volta, questo richiede una spesa sia dal punto di vista computazionale che da quello temporale. Questo problema viene facilmente risolto utilizzando una rappresentazione che sia naturale espressione delle simmetrie del sistema, in questo modo non sarà necessario questo step di training rendendo il tutto più semplice.

Generazione della mappa di densità

Il primo step è quello di costruire la densità atomica associata ad un determinato Z tramite una funzione gaussiana. Questo ci permette di passare da una forma discreta ad una continua. Questo si ottiene applicando la seguente:

$$\rho_Z(\vec{r}) = \sum_{\alpha}^Z e^{-\frac{|\vec{r}-\vec{r}_{\alpha}|^2}{2\sigma^2}} \quad (1.1)$$

quello che stiamo facendo è considerare un atomo con Z fissato qualunque e lo consideriamo come l'origine di un sistema di coordinate e per ogni singolo atomo α con lo stesso numero Z costruiamo una gaussiana. Nella formulazione sopra indicata r_{α} indica la distanza tra il centro dell'atomo α vicino e quello centrato con il sistema di riferimento definito sopra. Questa possiede quindi un picco in presenza dell'atomo. Mentre σ viene utilizzato per definire lo zoom con cui analizziamo il sistema, minore è il valore di σ più il risultato sarà perturbato anche da minime modifiche alla struttura, come una leggera modifica alla lunghezza dei legami (e conseguentemente della posizione relativa).

Questo step viene ripetuto per ogni singolo atomo, ottenendo come risultato che ogni atomo ha una mappa di densità dei suoi vicini ad esso associata.

Questo primo step ci permette di slegarci dal sistema di coordinate, usando infatti solo informazioni riguardanti la posizione relativa rendiamo il tutto invariante per traslazioni. La somma su tutti gli atomo con il medesimo Z rende inoltre invariante la rappresentazione rispetto allo scambio di due atomi uguali. La nuvola che viene così creata non è invariante per rotazioni in quanto se il centro del sistema di riferimento temporaneo menzionato sopra non è simmetrico per rotazione allora non lo sarà neanche la nuvola risultate.

Nel caso siano presenti elementi con diverso numero atomico, come non di rado succede, la mappa di densità viene separata in base ad esso, quindi nel caso avessimo Z_1 e Z_2 allora avremmo due mappe distinte, ρ_{Z_1} e ρ_{Z_2} . Questo mantiene l'invarianza per permutazione e permette anche di distinguere le diverse specie chimiche presenti.

Vogliamo dare adesso una rappresentazione grafica di questo concetto, facciamo riferimento alla seguente immagine 1.2:

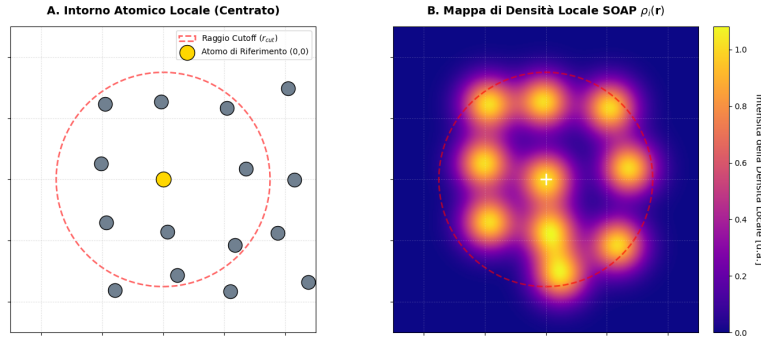


Figura 1.2: Rappresentazione generata tramite matplotlib del primo step della procedura SOAP

questa immagine è il risultato della somma di tutte le singole mappe prodotte per gli atomi interni al r_{cut} , dove è stato preso come riferimento l'atomo color oro posto al centro del sistema di coordinate. Il risultato finale si ottiene prendendo come atomo d'oro, a turno, ognuno degli atomi presenti. Per semplicità di rappresentazione la struttura atomica è stata generata come rumore di un cristallo periodico, questo non rappresenta nessuna struttura particolare, è solo una semplice griglia sulle cui intersezioni è stato posizionato un generico atomo.

Il controllo che viene dato dal parametro σ della gaussiana è rappresentato nella seguente 1.3:

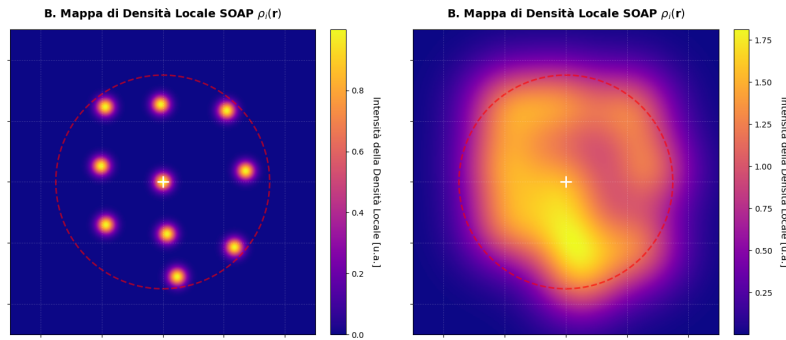


Figura 1.3: Rappresentazione generata tramite matplotlib del primo step della procedura SOAP per diversi valori del parametro σ

dove il pannello a sx rappresenta il caso di una σ eccessivamente piccola e quello a dx il caso di una σ eccessivamente grande. In entrambi i casi è possibile notare come l'informazione che se ne ricava non è adeguata agli scopi descritti sopra. Infatti come detto vogliamo che l'aggiunta di una piccola quantità di rumore non modifichi in modo sensibile il fattore di somiglianza tra le due strutture, nel primo caso la minima variazione di posizione porta a decretare che gli ambienti sono sensibilmente diversi, mentre nel secondo caso anche una decisa modifica delle posizioni porta alla conclusione che i due ambienti siano simili.

Per quanto riguarda il valore di r_{cut} , esso controlla il significato che diamo ad "atomi vicini". Se esso è troppo piccolo si perdono informazioni riguardanti la struttura in esame, mentre se troppo grande arriviamo ad avere un vettore di feature complesso e pesante che non fornisce particolare vantaggi.

Espansione in funzioni di base

In questo punto ci poniamo di trovare un metodo che ci permetta di riassumere in modo compatto una funzione continua come quella rappresentata dalla mappa di densità. Per fare questo ci procuriamo delle funzioni di base, nel nostro caso radiali e angolari.

L'espansione in serie per la mappa di densità è data da:

$$\rho_Z(\vec{r}) = \sum_{n=1}^{n_{max}} \sum_{l=1}^{l_{max}} \sum_{m=-l}^l c_Z(;n, l, m) g(r; n) Y(\theta, \phi; l, m) \quad (1.2)$$

dove il ; è posto a separatore tra le variabili e i parametri da cui dipendono le funzioni. Il coefficiente c_Z è per la relativa specie in considerazione, come indicato sopra in presenza di specie diverse si costruiscono mappe separate.

In particolare si ha che:

$$c_Z(;n, l, m) = \int_{\mathbb{R}^3} dr d\theta d\phi g(r; n) Y(\theta, \phi; l, m) \rho_Z(\vec{r}) \quad (1.3)$$

dove le $g(r; n)$ sono funzioni radiali e $Y(\theta, \phi; l, m)$ sono armoniche sferiche. Quello che stiamo andando a determinare tramite il coefficiente c_Z è la "misura" della nostra mappa di densità lungo una determinata componente di base, in questo caso funzioni radiali e armoniche sferiche (si utilizzano queste funzioni per le loro proprietà matematiche). Possiamo quindi adesso utilizzare questi coefficienti (un semplice vettore numerico) per ricostruire un oggetto tridimensionale come lo è la mappa di densità.

In questa sezione non ci dilunghiamo sui dettagli della forma di queste funzioni.

Lo scopo di questo step è quello di semplificare la rappresentazione, come detto passiamo da una struttura tridimensionale complessa ad un singolo vettore numerico. Non abbiamo ancora aggiustato le restanti invarianze, in quanto alla fine di questo step manca ancora l'invarianza per rotazione, la sua risoluzione sarà il cardine dello step successivo.

Calcolo del power spectrum

Per rendere la nostra rappresentazione andiamo a calcolare il power spectrum parziale, ottenuto dalla seguente equazione:

$$p_{Z_1 Z_2}(n, n', l) = \pi \sqrt{\frac{8}{2l+1}} \sum_{m=-l}^l \overline{c_{Z_1}(n, l, m)} c_{Z_2}(n', l, m) \quad (1.4)$$

dove il primo coefficiente è sotto complessa coniugazione, nel caso si usino solo armoniche sferiche reali, diventa superfluo. La presenza di Z_1 e Z_2 indica che questo viene calcolato per ogni coppia unica (indipendentemente dall'ordine) di specie chimiche presenti. Nel contesto del dataset A, quello che si produce è (Z_{Ta}, Z_{Ta}) , (Z_O, Z_O) e (Z_{Ta}, Z_O) .

Senza entrare nei dettagli, diciamo che questo processo rende il risultato invariante dal punto di vista delle rotazioni in quanto le rotazioni possono essere rappresentate tramite matrici unitarie (oppure ortogonali nel caso reale), il processo descritto sopra "trasforma" la matrice di rotazione (con cui possiamo rappresentare la rotazione della mappa di densità rispetto ad un qualunque sistema di riferimento) nella matrice identità, che quindi non modifica il valore del coefficiente.

Le correlazioni che si trovano per valori di Z_1 e Z_2 diversi servono per tenere traccia delle posizioni relative delle diverse specie atomiche.

Risultato finale

Capitolo 2

Discussione teorica sulle librerie python utilizzate

2.1 ASE

Questa libreria [1] ci permette di analizzare le simulazioni di dinamica molecolare prodotte tramite QE. Essenzialmente possiamo dire che svolge il ruolo di ponte tra la struttura fatta di coordinate e una che sia computazionalmente utilizzabile. Ci permette di trasportare le informazioni che QE utilizza per svolgere i calcoli, come le condizioni di periodicità, in un formato poi accessibile anche da altre librerie.

In particolare per QE abbiamo utilizzato i seguenti:

Listing 2.1: Snippet usato per il dataset A

```
1 frame = ase.io.read(file_name, index=":", format="espresso-out")
```

dove `index=":"` viene utilizzato per leggere tutti i frame presenti nel file di input e `format="espresso-out"` serve per impostare la tipologia di file da leggere.¹

La funzione `read` ci restituisce un oggetto di tipo `atoms`², che costituisce l'oggetto python che contiene le informazioni che definiscono il materiale da simulare che sono state estratte dai file di output di QE.

2.2 Dscribe

Questa è la libreria [2] che si occupa di trasformare la struttura atomica definita tramite ASE in una qualcosa comprensibile dai modelli di ML. Lo scopo principale di questa procedura è quello di rendere il training di un modello il più facile possibile. L'idea è quella di creare un oggetto che identifichi in modo univoco una determinata struttura atomica, e che sia invariante per modifiche "matematiche" della struttura stessa, come possono esserlo le rotazioni; se così non fosse il modello dipenderebbe da un sistema di coordinate e andrebbe sottoposto a training anche per le diverse rotazioni. Questo processo porta alla produzione di un descriptor, o vettore di feature che contiene l'informazione sulle invarianze delle leggi fisiche che descrivono il sistema. La prima cosa che facciamo notare è che separando il descriptor dalla fase di training è possibile riutilizzarlo con un diverso modello, diminuendo così il tempo necessario per l'addestramento. Abbiamo deciso di utilizzare il descriptor SOAP, per la rappresentazione locale della struttura atomica in analisi. [3, 4]

¹La specifica sezione della documentazione riguardante questo punto è reperibile al seguente <https://ase-lib.org/ase/io/io.html>

²La specifica sezione della documentazione riguardante questo punto è reperibile al seguente <https://ase-lib.org/ase/atoms.html>

Riportiamo adesso l'uso che abbiamo fatto di questo pacchetto:

Listing 2.2: Snippet usato per il dataset A

```

1  soap_obj = SOAP(
2      species=["Ta", "O"],
3      periodic=True,
4      r_cut=5.0,
5      n_max=8,
6      l_max=6
7  )
8
9  features = soap_obj.create(QE_data, n_jobs=-1)

```

dove `soap_obj` è interpretabile come una configurazione per il SOAP, `species` definisce tutti gli atomi che saranno incontrati nella struttura in esame, `periodic` serve per impostare l'uso della struttura periodica (precedentemente definita in ASE), `r_cut` definisce un cutoff per la rappresentazione locale (espresso in Å), `n_max` è il numero di funzioni radiali di base, `l_max` il massimo grado delle armoniche sferiche.³

In particolare l'accuratezza della rappresentazione, come il numero delle feature cresce proporzionalmente al valore di `n_max` e `l_max`.

La funzione `create` si occupa poi di creare il vero e proprio SOAP, in input viene passato `QE_data`, ovvero la struttura atomica in analisi definita tramite la libreria ASE.

Il risultato di questi calcoli è

Note sulla struttura matematica

DScribe in particolare per le armoniche sferiche usa la loro versione puramente reale, in modo da evitare l'utilizzo dell'algebra complessa, e come funzioni radiale delle GTO (gaussian type orbitals) in modo da rendere il tutto computazionalmente veloce e accurato.

³La dedicata documentazione è reperibile al seguente link <https://singroup.github.io/dscribe/latest/tutorials/descriptors/soap.html>

Bibliografia

- [1] *ASE documentation*. URL: <https://ase-lib.org/>.
- [2] *DScrive documentation*. URL: <https://singroup.github.io/dscribe/latest/>.
- [3] Lauri Himanen, Marc O.J. Jäger, Eiaki V. Morooka, Filippo Federici Canova, Yashasvi S. Ranawat, David Z. Gao, Patrick Rinke e Adam S. Foster. “DScrive: Library of descriptors for machine learning in materials science”. In: (2019).
- [4] Jarno Laakso, Lauri Himanen, Henrietta Himm, Eiaki V Morooka, Marc OJ Jäger, Milica Todorović e Patrick Rinke. “Updates to the DScrive library: New descriptors and derivatives”. In: *The Journal of Chemical Physics* 158.23 (2023).

-